



单目深度与点云重建

机器视觉实验课

叶书豪

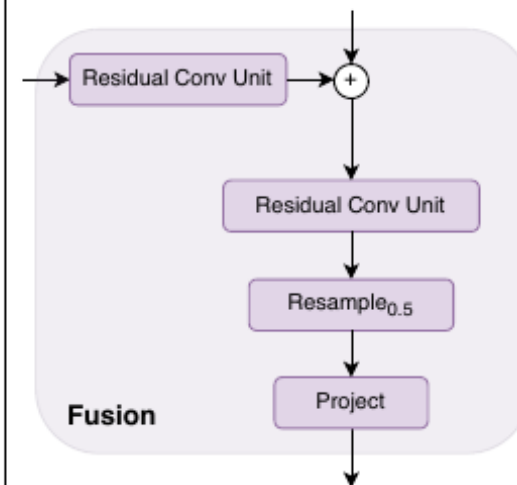
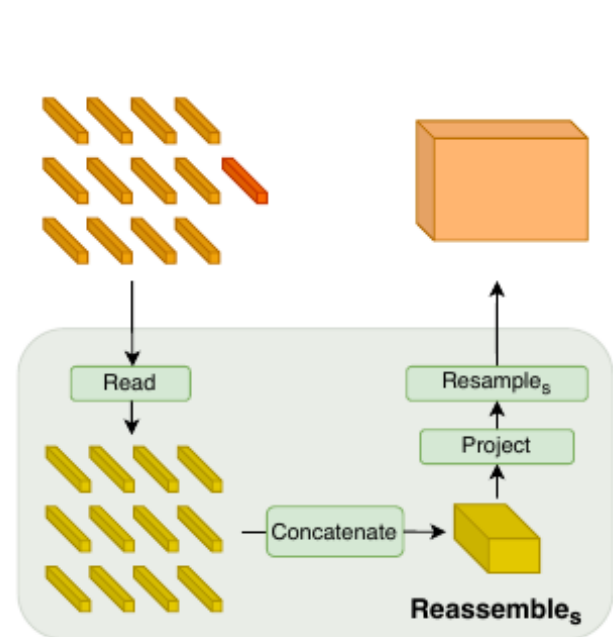
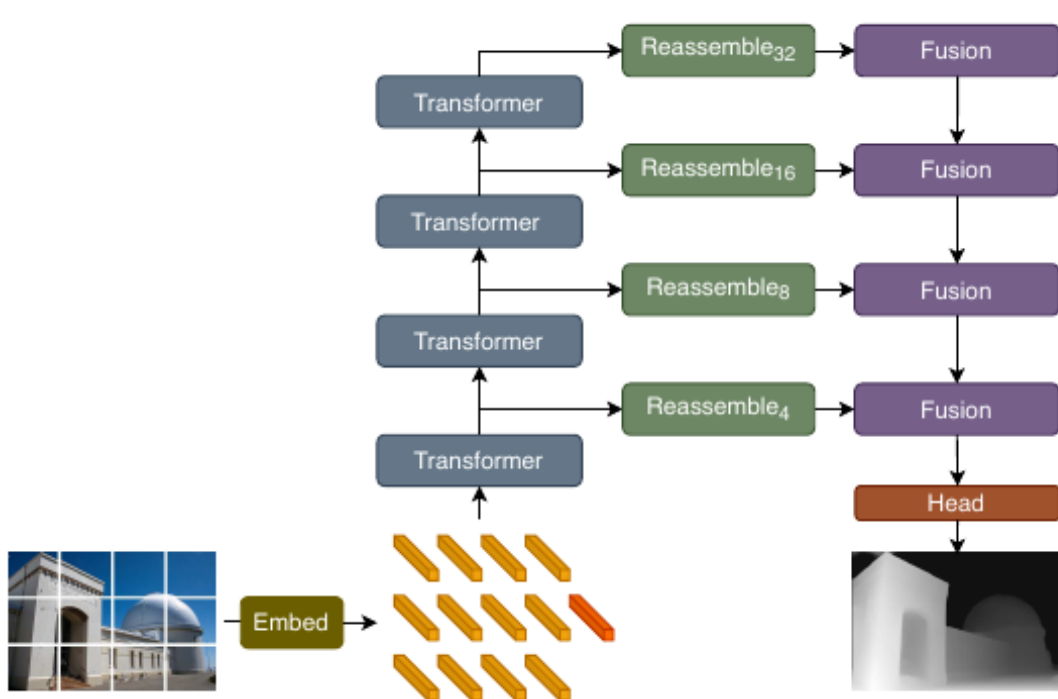
单目深度与点云重建

- 1 Depth Anything V2 介绍、安装
- 2 实验流程
- 3 实验作业

[GitHub: DepthAnything / Depth-Anything-V2](#)

[Paper: Depth Anything V2 \(NeurIPS 2024\)](#)

输入单张 RGB 图像，输出稠密相对深度。实验采用 Small，约 25M 参数。



DPT 结构示意图 (ICCV 2021)。本实验的编码器采用 DINOv2 ViT-S/14。



网络结构

1. DINOv2 特征提取

图像划分为 14×14 的 patch，
编码为 384 维特征。

ViT-S 包含 12 个 Transformer 层，
提取第 3、6、9、12 层输出。

不同层的特征包含纹理、边界
和场景结构信息。

2. DPT 深度预测

将序列还原为空间特征图，
构建不同分辨率的特征。

通道投影为 48、96、192、384，
统一为 64 通道后逐级融合。

卷积头输出单通道预测，
再插值到原图大小。

本实验使用预训练模型进行推理，无需训练网络。



相对逆深度与尺度、偏移对齐

记模型输出（逆深度）为 q ，真实深度为 z 。模型输出没有米制单位。

Ω 为整张图的全部有效像素，求一组共用的 a 、 b ：

$$(a^*, b^*) = \arg \min_{a, b} \sum_{i \in \Omega} \left(a q_i + b - \frac{1}{z_i} \right)^2$$

对齐后恢复深度：

$$\hat{z}_i = \frac{1}{a^* q_i + b^*}$$

在同一有效域内计算对齐后的误差。真实深度已提供尺度信息，因此结果反映对齐后的几何质量，不能解释为独立预测绝对距离的精度。



深度、内参与三维点云

令 Z 为对齐后的预测深度，使用内参将像素 (u, v) 反投影到相机坐标系：

$$X = \frac{(u - c_x)Z}{f_x} \quad Y = \frac{(v - c_y)Z}{f_y}$$

$$P = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = ZK^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

用预测深度生成 P ，用参考深度生成 G ，两者使用同一内参 K 。



对齐后的深度与点云误差

在 Ω 中比较相同像素。N 为有效像素数，点云保持原相机坐标系。

$$\text{AbsRel} = \frac{1}{N} \sum_{i \in \Omega} \frac{|\hat{z}_i - z_i|}{z_i}$$

$$\text{RMSE}_z = \sqrt{\frac{1}{N} \sum_{i \in \Omega} (\hat{z}_i - z_i)^2}$$

$$\text{RMSE}_{\text{XYZ}} = \sqrt{\frac{1}{N} \sum_{i \in \Omega} \|P_i - G_i\|^2}$$

AbsRel 无量纲；两项 RMSE 的单位为米，数值越小越好。



环境安装

安装 Miniconda 与 Git, 在终端中依次执行以下命令:

```
conda create -n mono_depth python=3.12
conda activate mono_depth
git clone https://github.com/DepthAnything/Depth-Anything-V2
cd Depth-Anything-V2
git checkout a561b849
```

```
pip install torch==2.7.1 torchvision==0.22.1 -i https://download.pytorch.org/whl/cu128
```

```
pip install -r requirements.txt numpy==1.26.4 opencv-python==4.10.0.84 matplotlib==3.9.4
```

```
pip install open3d==0.19.0
pip check
```



1. 下载预训练权重与实验数据

模型权重: [Depth-Anything-V2-Small \(官方 Hugging Face\)](#)

下载 depth_anything_v2_vits.pth, 放入新建的 checkpoints 文件夹。

实验数据: nyu_data.zip, 解压后将 nyu_data 放在仓库内。

```
Depth-Anything-V2/  
  run.py  
  checkpoints/depth_anything_v2_vits.pth  
  nyu_data/rgb/nyu_0001.png ...  
  nyu_data/reference/nyu_0001.npz ...
```

NPZ 保存 depth_m、K、valid。8 张图像均为 640×480 , 深度单位为米。



2. 保存原始预测并运行网络

在 run.py 中找到 infer_image, 在下一行增加 np.save:

```
depth = depth_anything.infer_image(raw_image, args.input_size)
np.save(os.path.join(args.outdir,
    os.path.splitext(os.path.basename(filename))[0] + '.npy'), depth)
```

随后运行:

```
python run.py --encoder vits --img-path nyu_data/rgb --outdir results_518 --pred-only
```

输出 PNG 用于观察相对远近; 同名 NPY 用于后续对齐、反投影和评估。

3. 观察深度图与输入分辨率

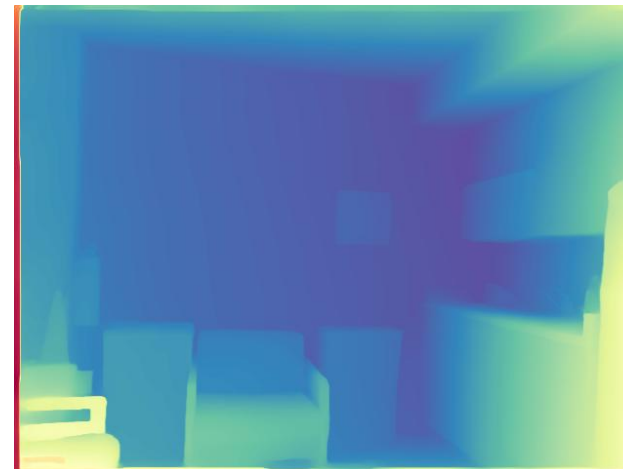
--input-size 控制网络输入尺寸。保持其他条件，改为 280 再运行：

```
python run.py --encoder vits --img-path nyu_data/rgb --outdir out280 --input-size 280 --pred-only
```

RGB 图像



相对深度显示图 (518)



比较运行时间、物体边界及细小结构。输出预测会恢复到原图尺寸。



4. 深度对齐 (geometry.py, 前半部分)

```
import sys, cv2, numpy as np, open3d as o3d
from pathlib import Path
sid, folder = sys.argv[1:3]
out = Path(folder)
q = np.load(out / (sid + '.npy')).astype(float)
d = np.load('nyu_data/reference/' + sid + '.npz')
g, K, valid = (d[k] for k in ['depth_m', 'K', 'valid'])
assert q.shape == g.shape and np.isfinite(q).all()
A = np.c_[q[valid], np.ones(valid.sum())]
a, b = np.linalg.lstsq(A, 1 / g[valid], rcond=None)[0]
assert a > 0 and np.isfinite(a + b)
inv = a * q + b
z = np.divide(1, inv, out=np.full_like(inv, 10), where=inv > 0)
z = np.clip(z, .1, 10)
np.save(out / (sid + '_depth_m.npy'), z)
```

在仓库根目录新建 geometry.py。

5. 点云生成与评估 (接上页)

```
v, u = np.indices(z.shape)
rays = np.stack([u, v, np.ones_like(u)], axis=-1)
rays = rays @ np.linalg.inv(K).T
P, G = z[..., None] * rays, g[..., None] * rays
e = z[valid] - g[valid]
print(sid, 'Aligned AbsRel =', np.mean(np.abs(e) / g[valid]))
print('Depth RMSE (m) =', np.sqrt(np.mean(e ** 2)))
xyz2 = np.sum((P[valid] - G[valid]) ** 2, axis=1)
print('XYZ RMSE (m) =', np.sqrt(np.mean(xyz2)))
rgb = cv2.imread('nyu_data/rgb/' + sid + '.png')[..., ::-1] / 255.
for name, points in [('pred', P), ('gt', G)]:
    cloud = o3d.geometry.PointCloud()
    cloud.points = o3d.utility.Vector3dVector(points[valid])
    cloud.colors = o3d.utility.Vector3dVector(rgb[valid])
    o3d.io.write_point_cloud(str(out / (sid + '_' + name + '.ply')), cloud)
```

运行指令: `python geometry.py nyu_0001 results_518`

6. 三维可视化

新建 view.py, 保存以下代码后运行 python view.py:

```
import open3d as o3d
p = o3d.io.read_point_cloud('results_518/nyu_0001_pred.ply')
o3d.visualization.draw_geometries(
    [p], front=[0, 0, -1], up=[0, -1, 0],
    lookat=p.get_center(), zoom=0.7)
```



预测点云



参考点云 (文件后缀改为 _gt)

拖动旋转、滚轮缩放。逐图运行评估, 记录三项指标, 并对比 518 与 280。



1. 两人一组，完成单目深度与点云重建实验，保证程序跑通

第三周实验课课上轮流检查

2. 每组撰写实验报告，实验设置、结果与分析，不要贴基础知识和代码， ≤ 5 页

ddl: 第五周实验课当天，上传到学在浙大

3. 评分细则（小组内成员分数一致）

80%：完成深度预测、点云生成与精度评估。记录 8 张图的指标及均值。

10%：比较 input-size=518 与 280，分析运行时间、深度细节和点云误差

10%：选用 Depth Anything V2 的 Metric Depth 版本进行对比。深度和点云精度有何变化？尺度信息起什么作用？如何保证比较条件公平？